

AI DLC三大核心引擎解读： TCRay+Xpark + Meson

AI DLC分布式计算引擎TCRay

从多模态数据处理、模型训练到 Agentic AI 的统一计算底座

数据湖计算边界正在扩展：从 SQL / Batch 到 AI Native Workload

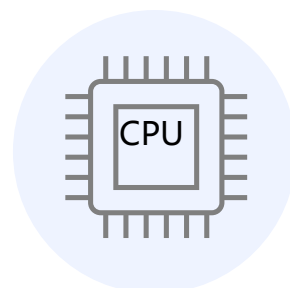


Agent 时代正在重塑数据平台的角色：数据湖不再是只作为服务人类分析决策的 ETL 与查询底座，而是需要升级为承载多模态处理、模型后训练、批量推理、在线服务与 Agent 编排的 AI Native 统一计算底座，让数据消费、轨迹沉淀、记忆生成与智能进化在同一基础设施上形成闭环。

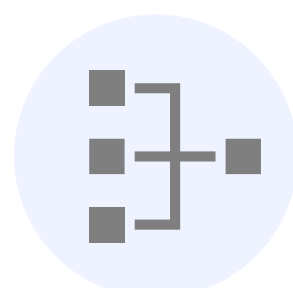
传统大数据计算引擎特点



大规模结构化数据处理



以 CPU 为核心



粗颗粒度调度

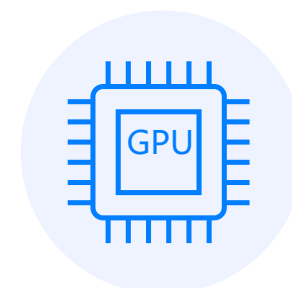


VS
范式鸿沟

AI Workload 特点



海量多模态数据



以 GPU 为核心



资源细颗粒度调度

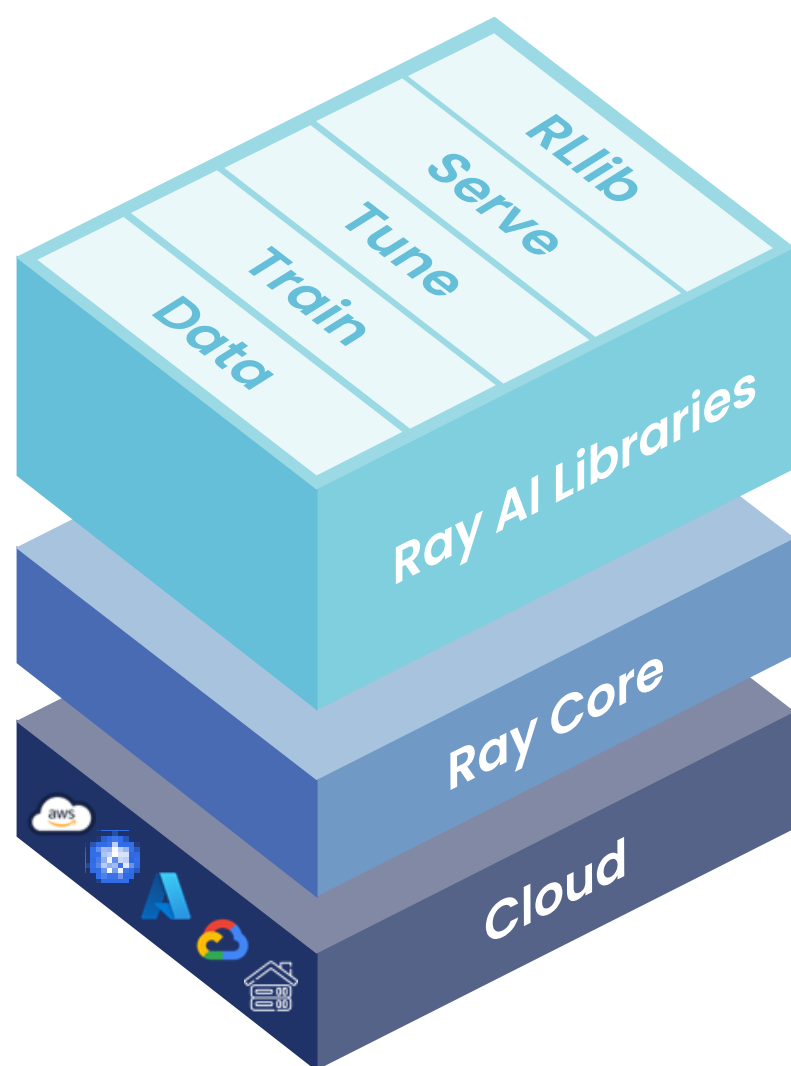


deepspeed

Ray 的核心定位：AI Native 全生命周期的统一运行时



Ray is an AI compute engine. Ray consists of a core distributed runtime and a set of AI Libraries for accelerating ML workloads.



● Python 原生分布式底座

贴近 AI 开发者生态，让 AI 应用可以用熟悉的 Python 方式规模化运行。

● Task + Actor + Object 统一抽象

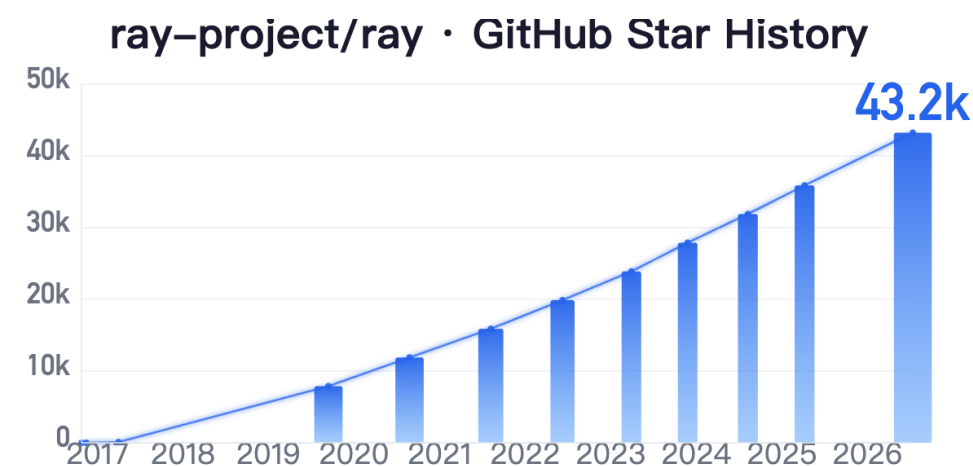
用一套编程模型覆盖无状态并行计算与有状态服务，降低分布式开发和异构资源调度复杂度。

● AI 全生命周期组件体系

从 Ray Core 到 Ray Data / Train / Tune / Serve，贯通数据处理、模型训练、参数调优、推理服务等关键链路，支撑统一 AI Infra 建设。

Ray 的核心价值，是把 **Python 原生开发体验**、**分布式运行时**和 **AI 全生命周期组件** 统一到同一个计算平台中。

Ray 产业趋势：从分布式运行时走向 AI Infra 标准组件

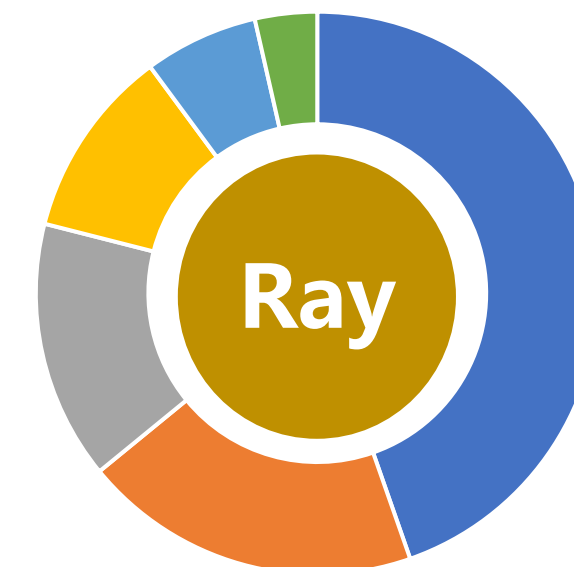


贡献者 **1500+ 人**



PyPI 下载 **6.9+ 亿次**

应用规模 · 从开源热度走向大规模使用



- veRL
- OpenRLHF
- Slime
- AReal
- ROLL
- NVIDIA-NeMo/RL

后训练生态 · 成为 RL / RLHF 框架的标准依赖

PYTORCH FOUNDATION PROJECTS

(2025.10.22)



DeepSpeed

LLM



PyTorch



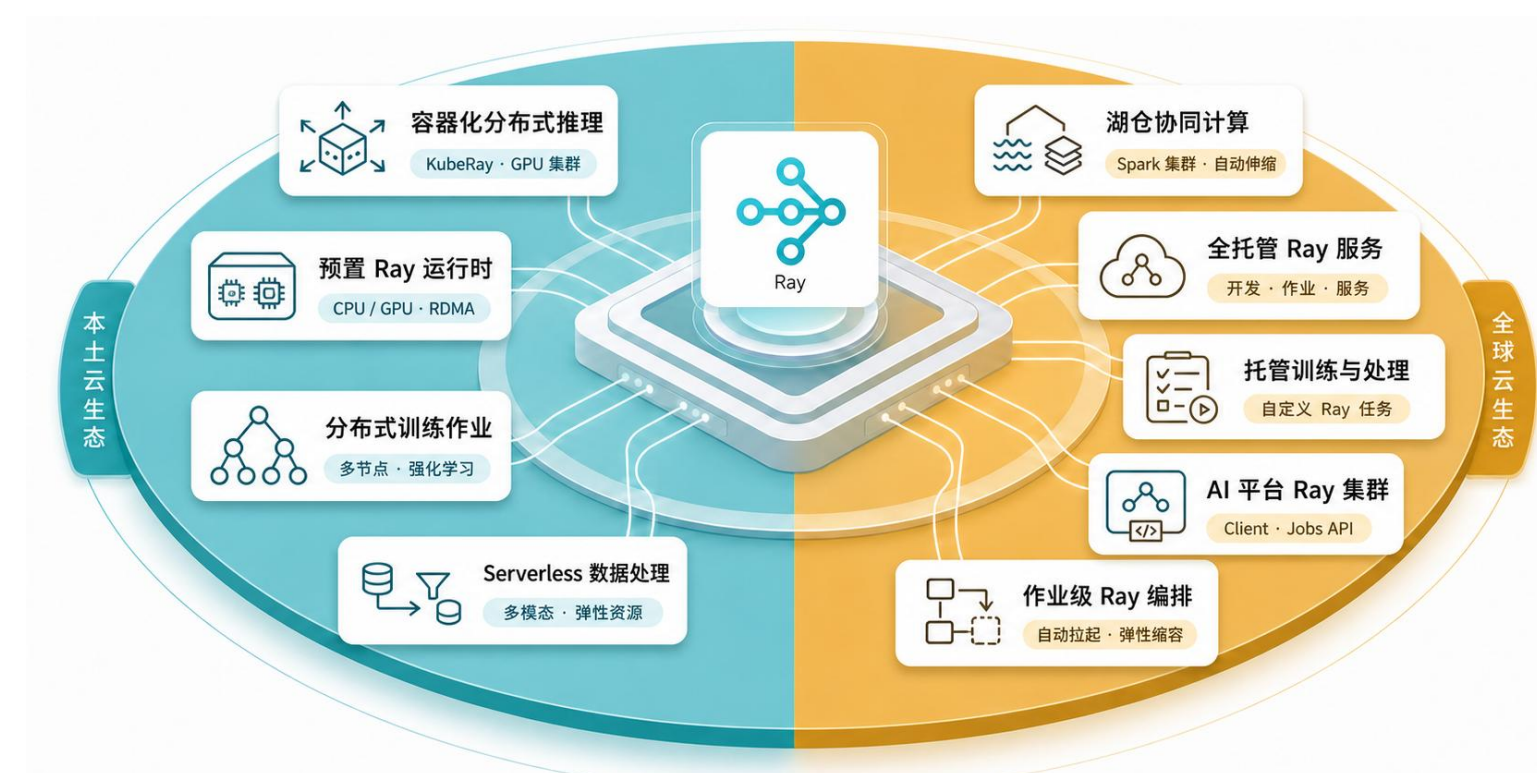
RAY

Safetensors



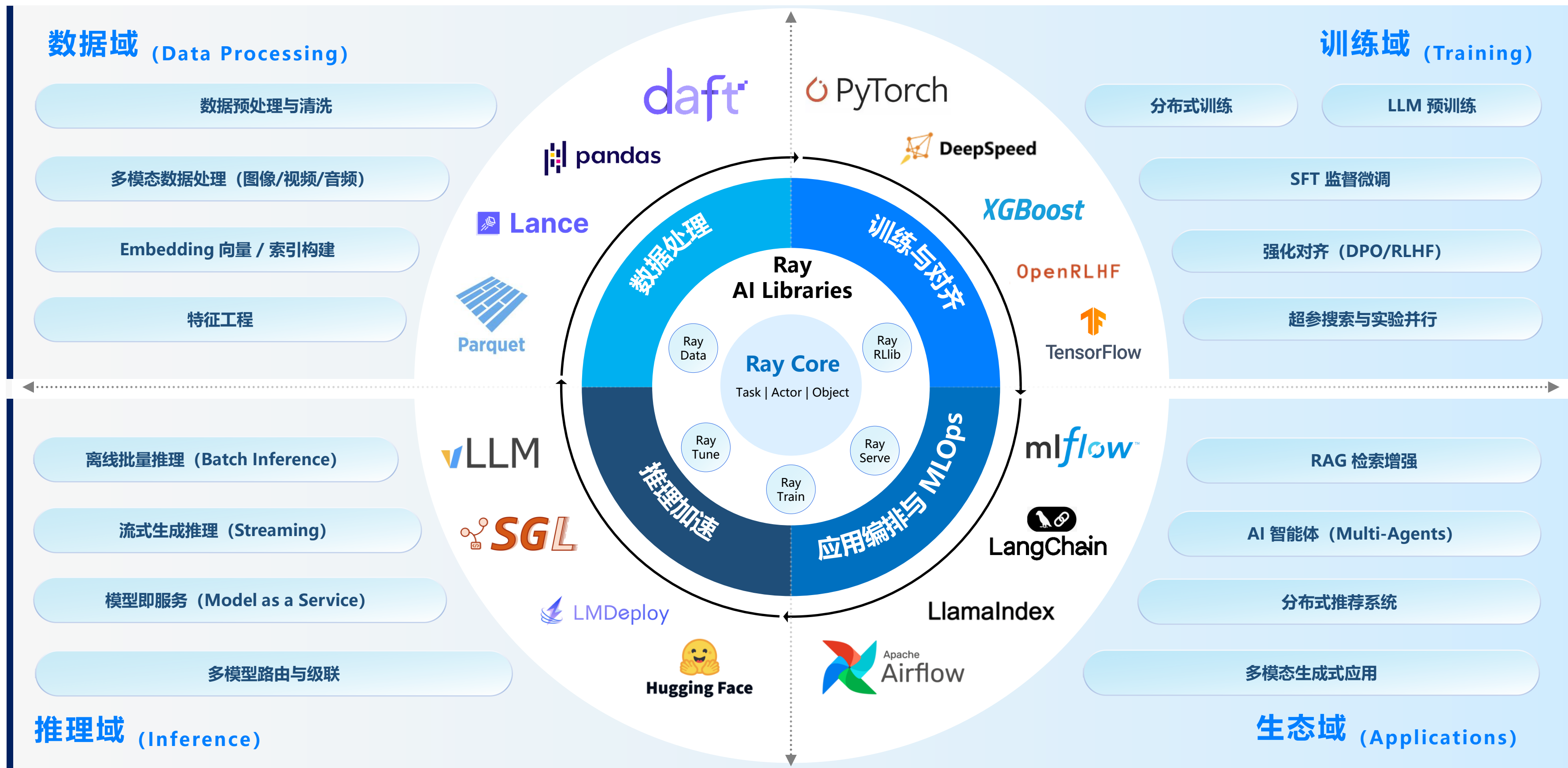
Helion

社区治理 · 从快速迭代走向企业级稳定



云厂商采纳 · 从开源框架走向托管服务

Ray 能力版图：贯通数据处理、模型训练、推理服务与 Agent 应用



| 开源 Ray 的生产化局限：企业级托管、治理与云原生能力待增强



托管服务

集群与作业管理 -> 自助式平台服务

KubeRay 已提供集群与作业生命周期、自动伸缩等基础能力。企业平台进一步统一作业入口、运行环境和版本管理，让用户无需直接管理底层集群。

✓ 统一作业入口

✓ 集群生命周期

✓ 环境与镜像

✓ 版本升级



多租户治理

逻辑资源调度 -> 租户级资源治理

Ray 的资源声明主要用于调度，并不等同于租户隔离。生产平台还需提供访问控制、配额与队列、优先级、资源隔离和租户用量统计，支持多团队共享算力。

✓ 访问控制

✓ 配额与队列

✓ 优先级调度

✓ 资源隔离



稳定性保障

基础可观测 -> SLA/SLO 保障

Ray 已提供日志、指标和 Dashboard，也可以对接 Prometheus。企业平台需要进一步统一多集群的监控与告警，保留历史作业状态，并补充故障诊断和自动恢复，形成可衡量的 SLA / SLO 保障。

✓ 统一监控告警

✓ 历史作业追溯

✓ 故障诊断

✓ 自动恢复



云产品集成

K8s 原生运行 -> 云上产品化交付

KubeRay 已支持 Kubernetes 原生部署。平台还需进一步打通云上的网络、存储、镜像和数据目录能力，让 Ray Workload 可以直接使用这些云资源，减少用户自行适配。

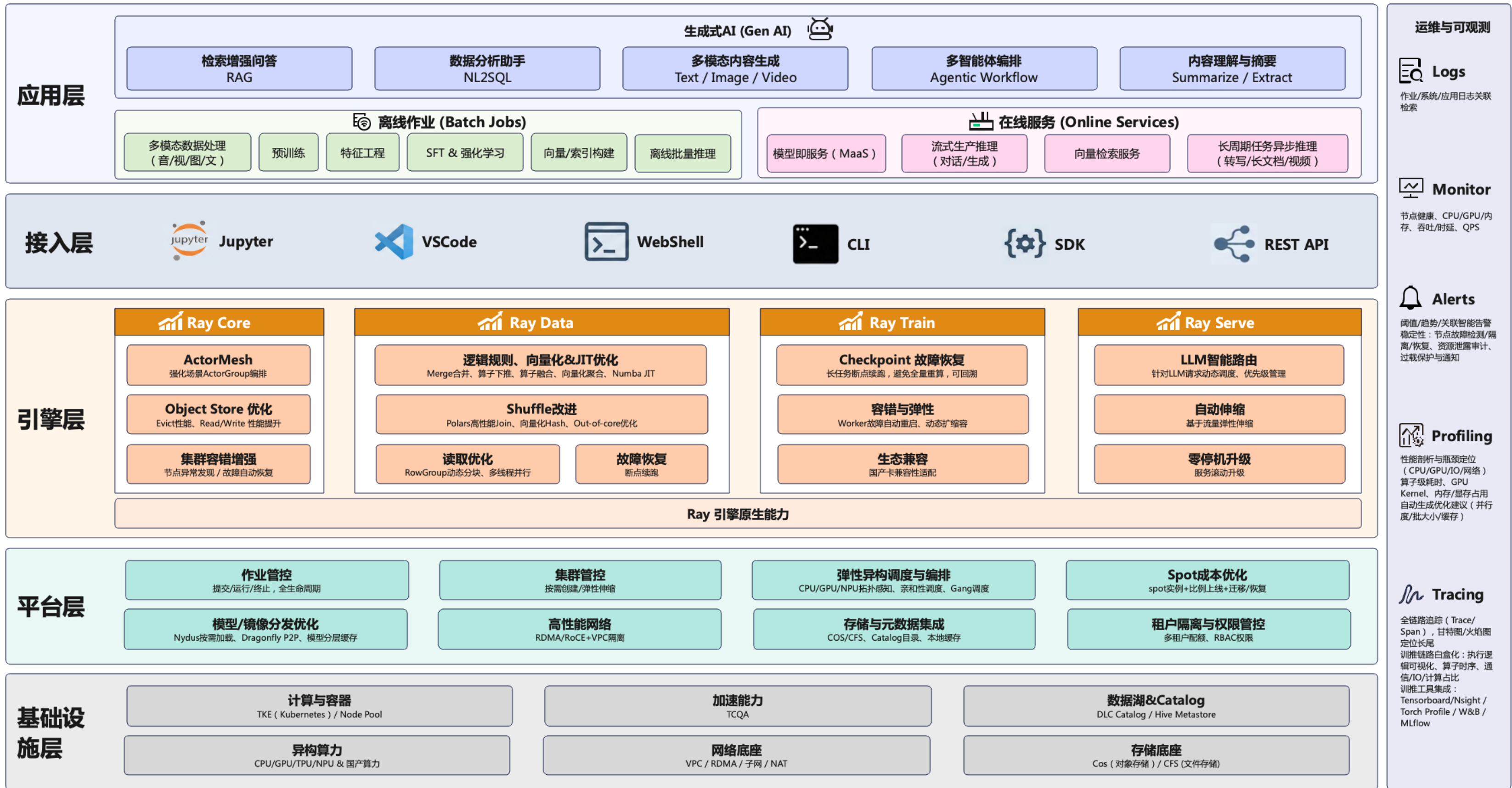
✓ VPC/RDMA网络

✓ COS/CFS/数据湖

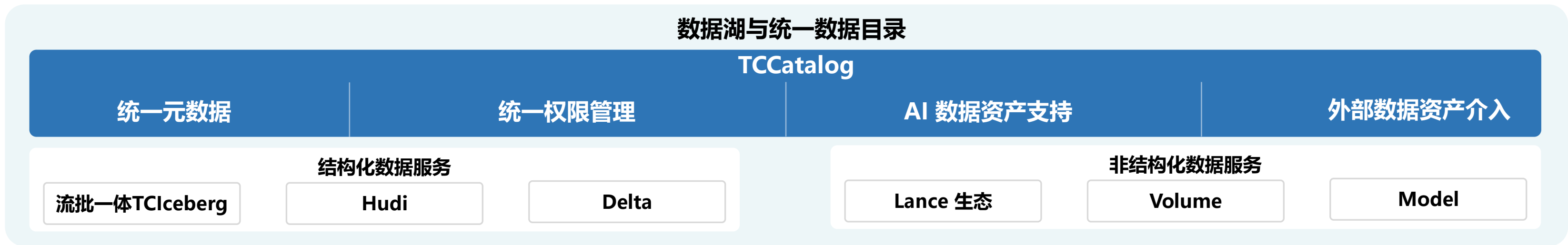
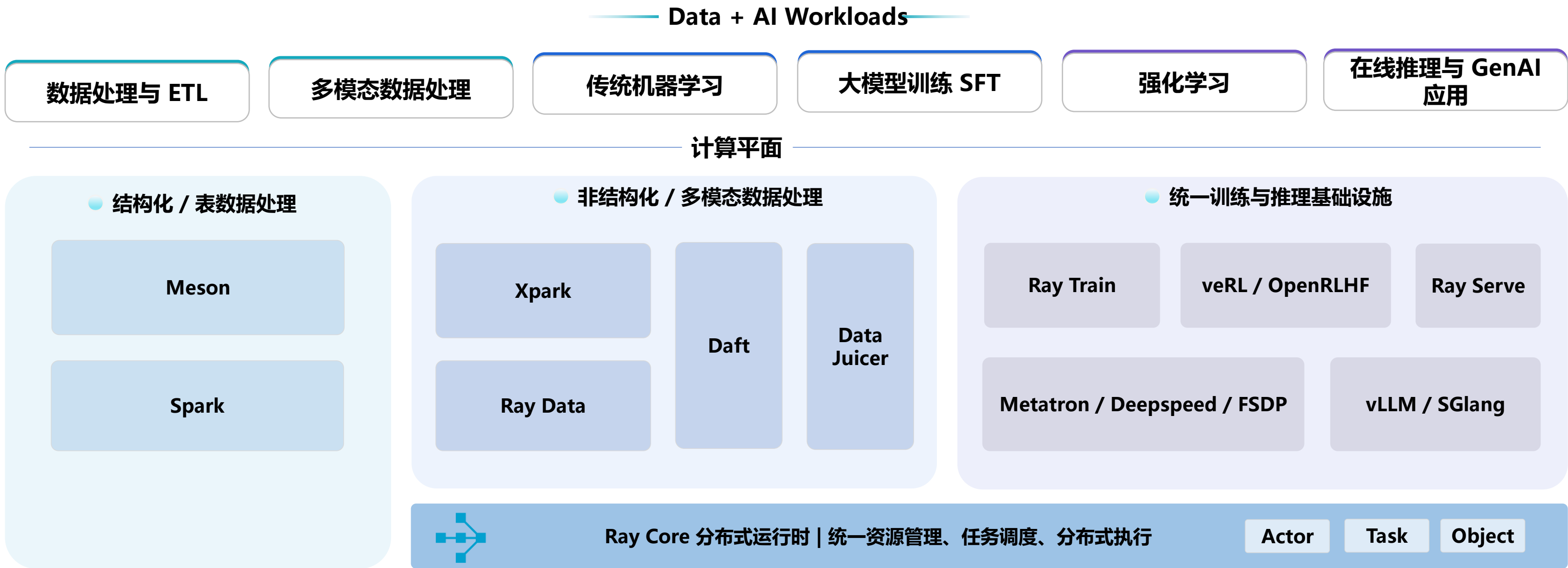
✓ 镜像仓库

✓ TCCatalog

TCRay 能力架构：面向生产级 Serverless Ray 的企业增强

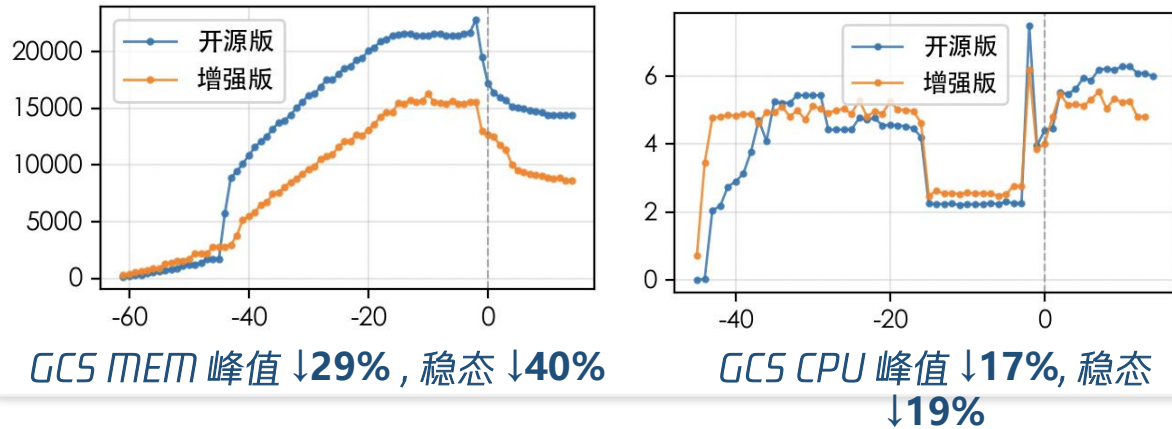


TCRay 在 AI DLC 中定位：连接数据湖与智能应用的AI计算平面

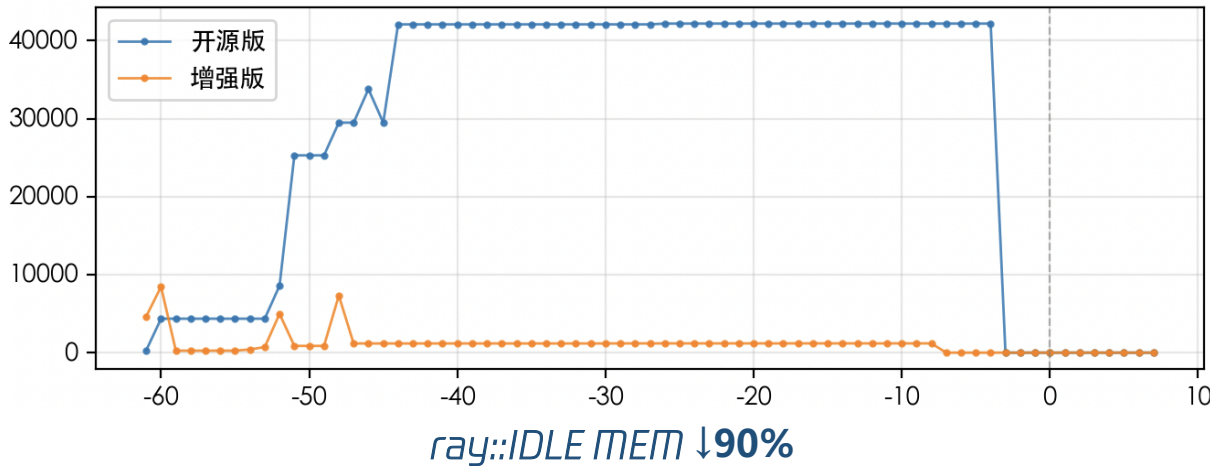


TCRay 建设进展：产品能力、技术增强、落地场景

规模化：3k Nodes / 10w Actors



稳定性：Worker OOM 优化



历史可观测：History Server 全面升级

加载耗时

↓98%

分钟级 → 秒级

峰值内存

↓99%

GiB → MiB

Job: 12h | 12GB Cos存储 | 1339w Events | 12w Tasks | 13k num logs

Event 分类存储

Task 事件按需加载

动态 LRU 驱逐

Logs 增量上传

高优信息分级加载

无效信息剪枝降噪

多租户

资源队列

弹性伸缩

流量控制

模型训练

高可用

标签鉴权

优先级调度

镜像加速

统一网关

模型服务

运维&可观测

平台能力

落地场景



智能驾驶



具身智能



零售电商



游戏



AI Coding

TCRay 演进方向：面向规模化 AI Workload 的云上统一计算底座



01 更多承载

统一承载更丰富的 AI Workload，持续适配国产异构算力与开放生态。

Workload · 算力 · 生态 · 场景

02 更快运行

端到端加速资源供给与任务执行，持续缩短作业从提交到完成的时间。

快启动 · 快调度 · 快执行 · 快扩缩

03 更好体验

将智能能力贯穿开发运维全流程，持续降低 Ray 的使用与运维门槛。

事前推荐 · 事中洞察 · 事后诊断

04 更省成本

以按需供给和全局资源优化，持续提升利用率、降低单位任务成本。

空闲归零 · 低价资源 · 提升利用率

统一承载更多 Workload，以更高效率、更低门槛和更优成本运行在云上

AI DLC 多模态计算引擎·Xpark

面向 AI 时代的腾讯云自研多模态计算引擎

01

—
项目背景

02

—
架构介绍

03

—
核心能力

04

—
Xpark on AI DLC

01

项目背景

AI 时代，数据处理正在从结构化计算走向多模态处理



背景：AI 时代，如何高效构建大规模的多模态数据处理能力？

1 业务背景

当前AI 应用场景爆发，多模态数据快速增长

根因：AI驱动下，可被计算的业务数据边界不断扩大

LAION-5B
公开图文数据集，包含约 58.5 亿图文对；

YouTube
每分钟上传超过 500 小时视频内容，可被AI深度理解、检索

随着大模型多模态理解和推理能力的成熟，海量沉睡的多模态数据得以被挖掘
这催生了巨大的多模数据处理/检索需求

- 全球数据量 2025 达 **213 ZB**，非结构化占比 **92%**，CAGR **21.4%**
- 非结构化管理市场 **92→285 亿美元**，CAGR **15%** [IDC 2025]

2 技术挑战

传统大数据处理引擎在多模态场景下失配

数据形态复杂



算子体系不足



异构资源协同困难

3 解决方案

Xpark 多模态处理引擎

高性能的多模态算子

异构资源的高效协同

统一的多模态处理接口

02

Xpark架构

Xpark 多模态数据处理架构



布局多模态 AI-Native 数据处理，打造 AI 处理数据基础设施

多模场景

LLM / VLM
训练数据准备

RAG 知识库
构建

自动驾驶

具身智能

日志处理

用户画像

用户意图

...

接口层

Xpark Dataset

多模计算

Xpark
自研多模数据
计算引擎

文本类

理解、清洗、过滤、去重...

图片类

识别、标注、模糊、转换...

视频类

转码、增强、采样、分段...

...

TCRay

TI-ONE

TokenHub

内置模型
[Embedding/CLIP]

第三方模型

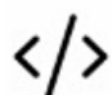
关键能力

1 高性能多模态算子

2 异构资源协同加速

3 分布式加速

多模数据



向量



文本



JSON

ES/向量数据库



图片



PDF



PPT



视频

对象存储/CFS/TCLake Volume



Iceberg



Paimon



Lance

TCLake Table

03

Xpark 关键能力

关键能力1：多模态数据处理算子

问题与挑战

- 1 **Spark 只具备比较基础的多模数据处理能力**
[如图片文件读取等, 复杂功能需适配 Python]
- 2 **Ray 多模态算子较少/未经优化, 部分性能比较差**
[如 `read_videos`]
- 3 **异构流水线往往需要用户实现和调优, 难度比较大**

提供简单易用的多模态算子, 可显著降低使用门槛, 提高开发效率

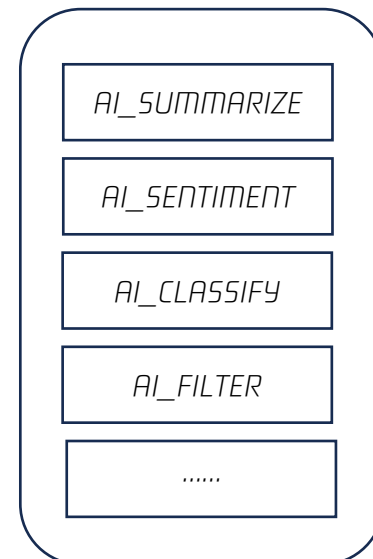
解决方案

Xpark 内置丰富多模态算子库, 优化高频场景

多模态算子



LLM AI Function

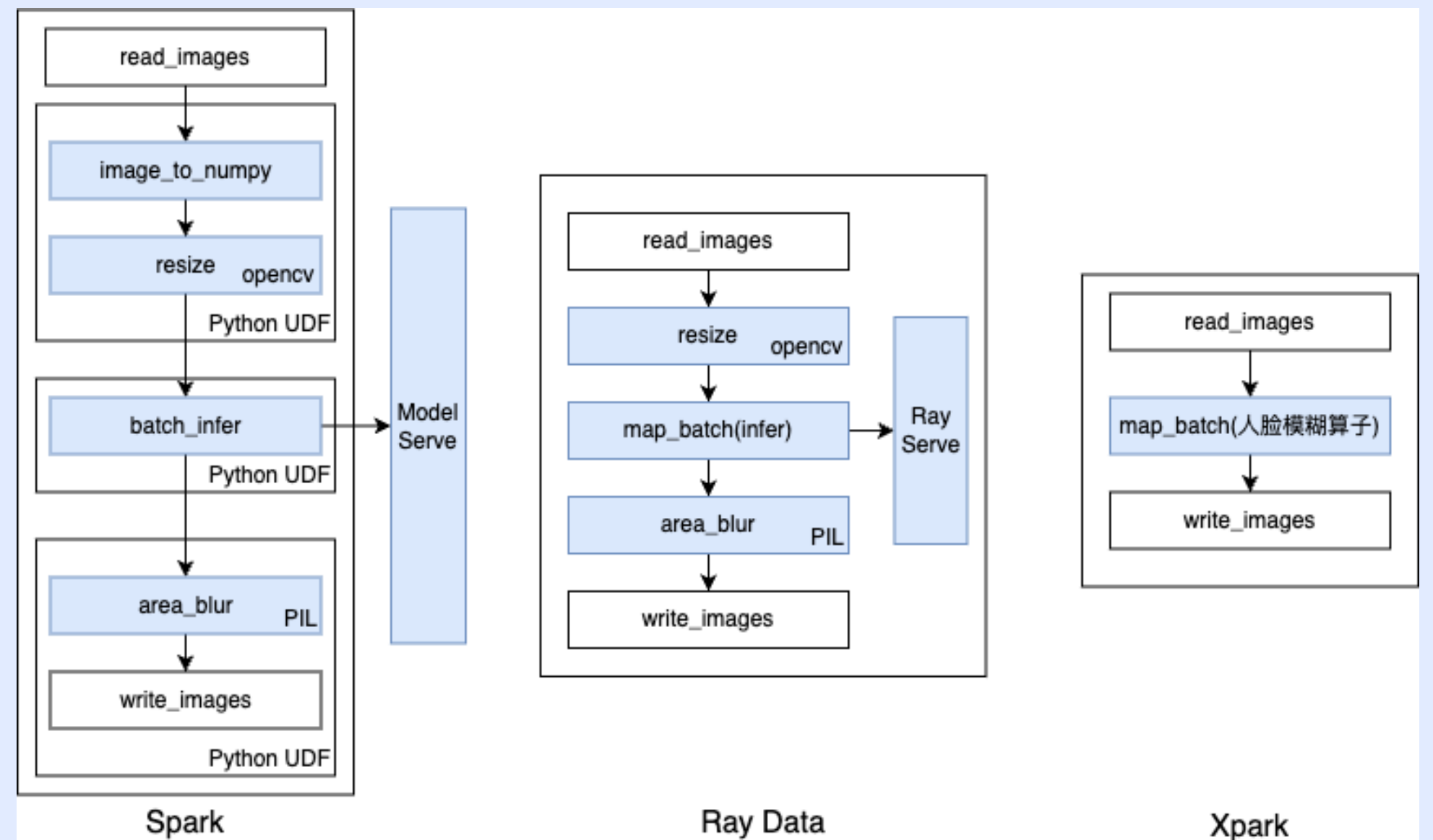


当前成果

集成 50+ 多模态算子, 大幅降低开发成本

通过模型级联, 判别式算子相比单纯大 LLM 调用节约 70% LLM 请求

支持 UDF, 可快速迁移已有业务逻辑



关键能力2：异构资源协同加速

问题与挑战

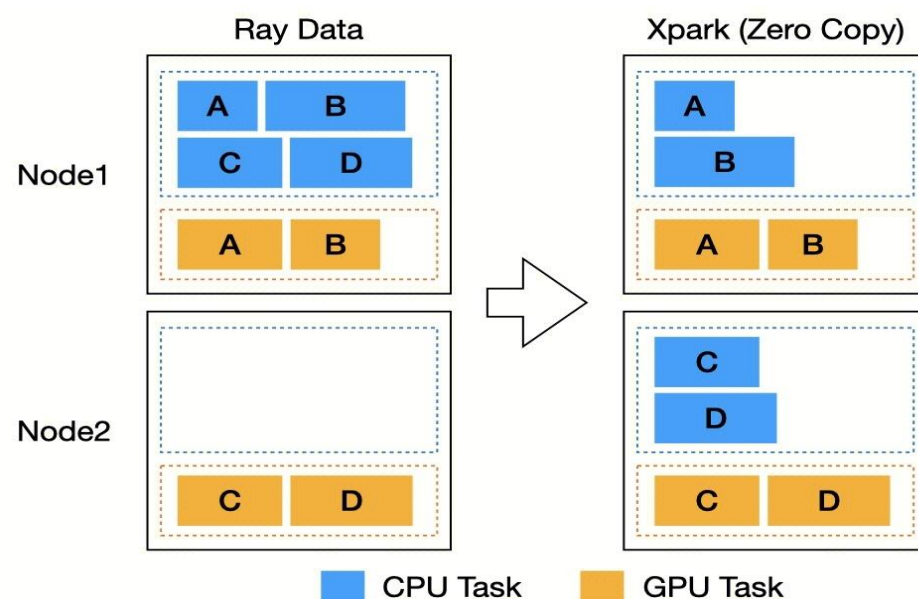
- 1 CPU/GPU 速率不匹配导致“流水线阻塞”
[GPU 可能等待 CPU 前置处理]
- 2 高带宽+分布式数据传输，带来的“通信墙”瓶颈
[多模态数据量大，分布式传输延时高]

异构硬件的高效协同和数据流转
是突破大规模多模计算性能瓶颈的关键点

解决方案

CPU/GPU 流水线 Overlap：异步化封装预处理、模型推理、后处理

减少跨机数据拷贝：CPU/GPU 协同调度，best-effort



当前成果

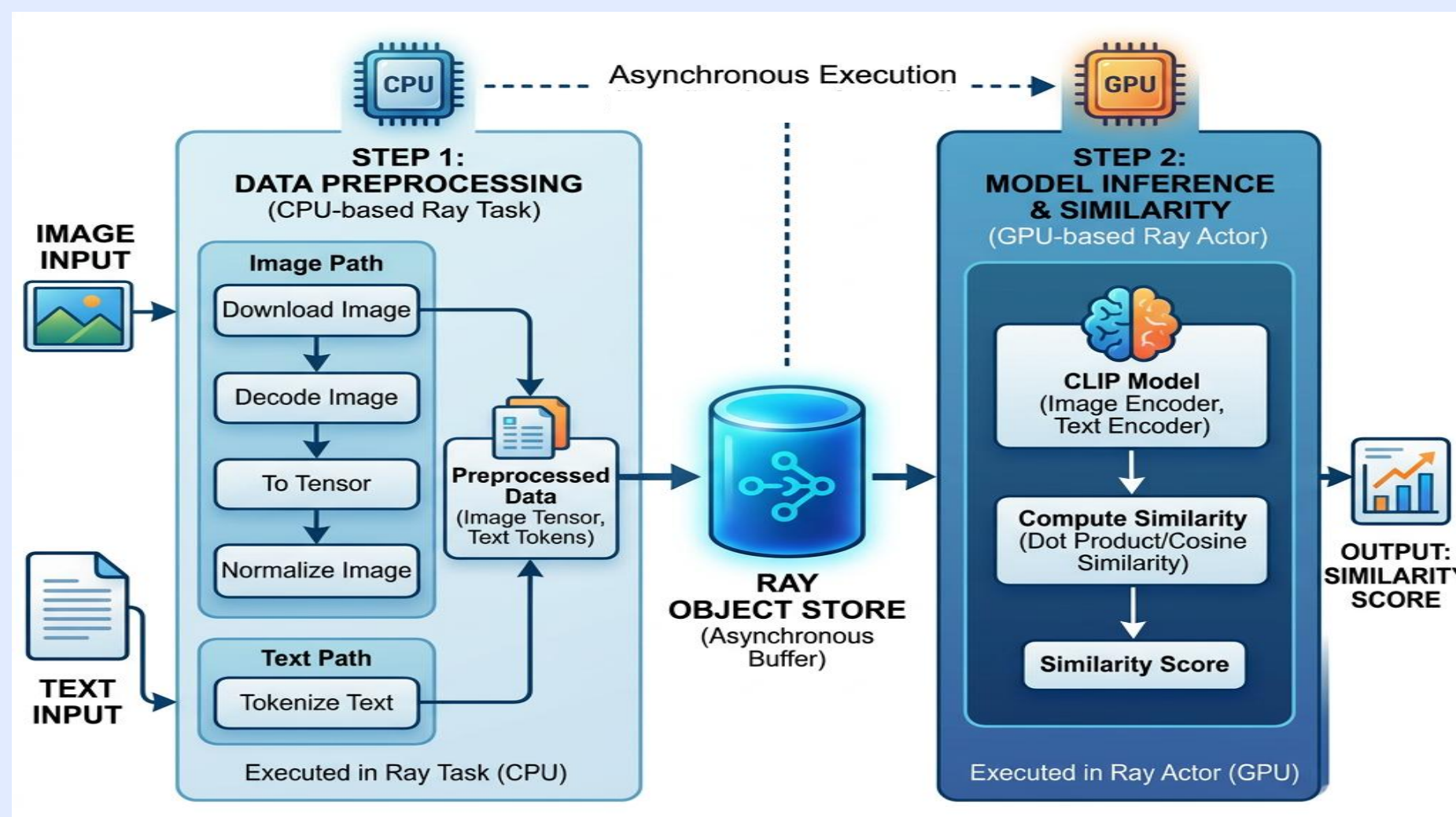
以 CLIP 图文匹配场景为例：

3 倍

推理吞吐相比开源 Data-Juicer
实现吞吐提升 3 倍

~100%

GPU 得到充分释放，使用率长
期稳定接近 100%



关键能力3：单机算子分布式化实现性能突破

问题与挑战

- 1 有状态单机算子往往较难实现分布式化，难以支撑大规模数据处理



A collection of text and code deduplication scripts

有状态单机算子需要合理高性能分布式实现

解决方案

以文本精确去重为例：有状态、分阶段、分布式，实现高性能去重

CPU-Intensive 关键路径使用 C++ 实现，解决 Python 性能问题

```
cdef class BTSUnionFind:
    """
    The original paper on BTS Union-Find is available at:
    https://ieeexplore.ieee.org/document/10598116
    """
    cdef:
        unordered_map[uint64_t, uint64_t] parent
        unordered_map[uint64_t, uint64_t] balance_changes
        uint64_t parallel_id
        uint64_t parallel_num
        uint64_t batch_size

    def __cinit__(self):
        self.parent = unordered_map[uint64_t, uint64_t]()
        self.balance_changes = unordered_map[uint64_t, uint64_t]()
```

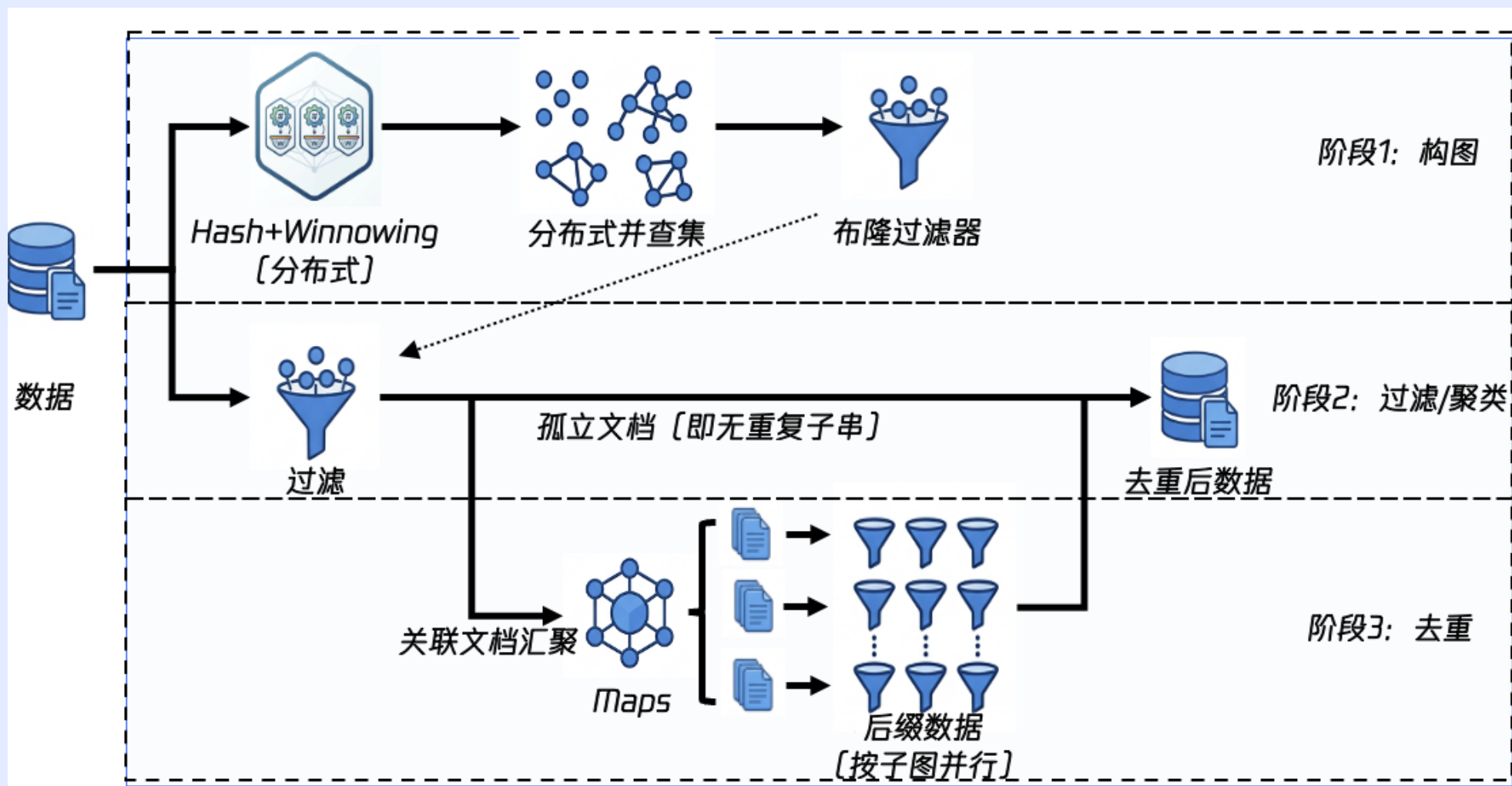
当前成果

高性能 Exact Substring Dedup 算子

- Text-dedup 开源单机算子快 **47.8 倍**
- C++ 实现的**高性能分布式 BTS**

高性能 Fuzzy Dedup 算子

- 开源 Data-Juicer[v1.4.2] **8.5 倍** 性能
- 基于元数据唯一 ID 生成，**省去一轮全量 IO**
- C++ 高性能 CJK 分词器



04 **Xpark on AI DLC**

Xpark on AI DLC 使用



数据实验室（调试开发/小批量验证）

WebIDE / JupyterLab

```
from xpark.dataset.expressions import col
from xpark.dataset import SpeechToText, from_items

ds = from_items(["multilingual.mp3"])
ds = ds.with_column[
    "text",
    SpeechToText[
        # Local transcriptions model.
        "Sysran/faster-whisper-large-v3",
    ]
    .options(num_workers={"GPU": 1})
    .with_column(col["item"]),
]
```

任务式提交（大规模运行）

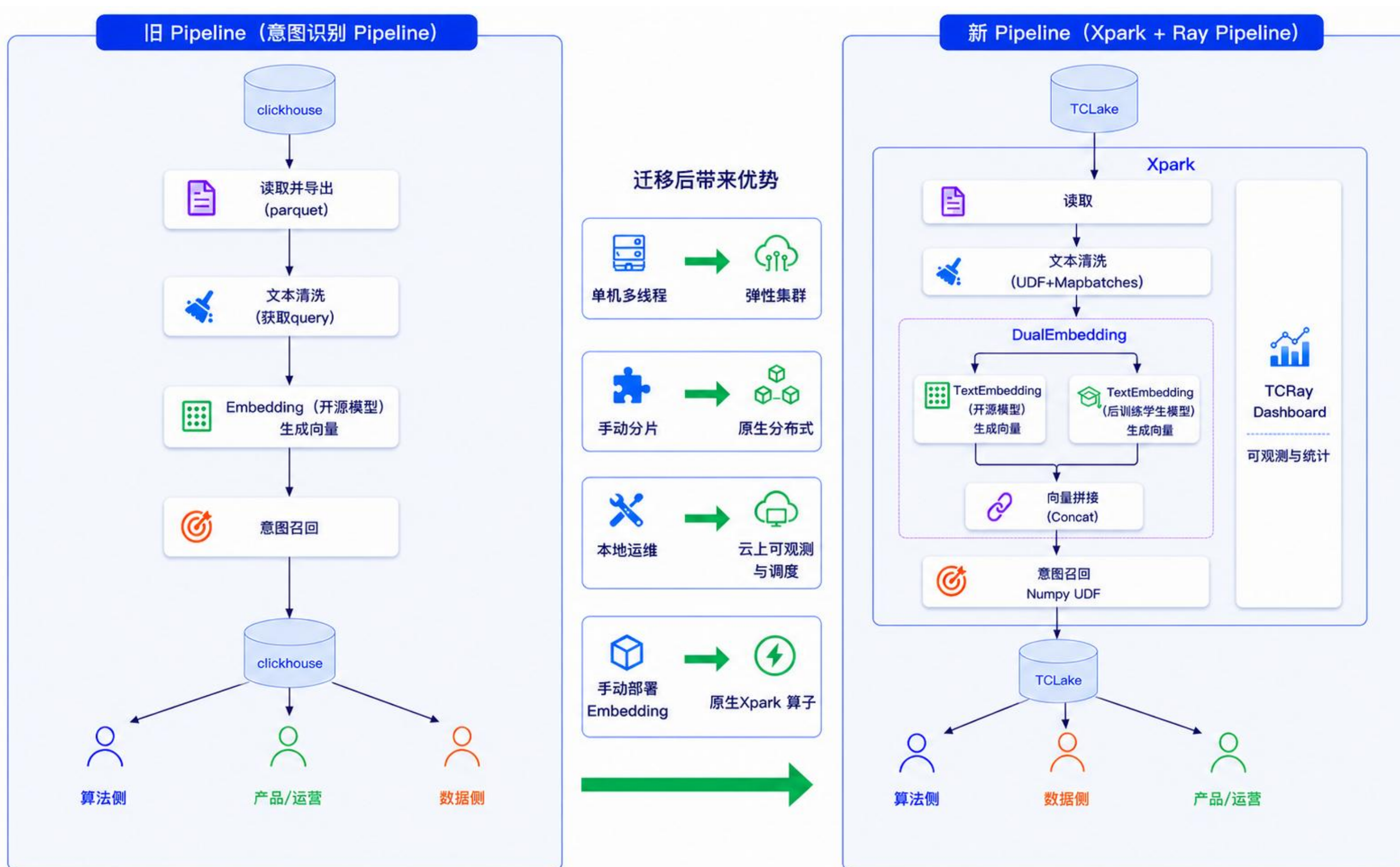
腾讯云控制台

数据湖计算

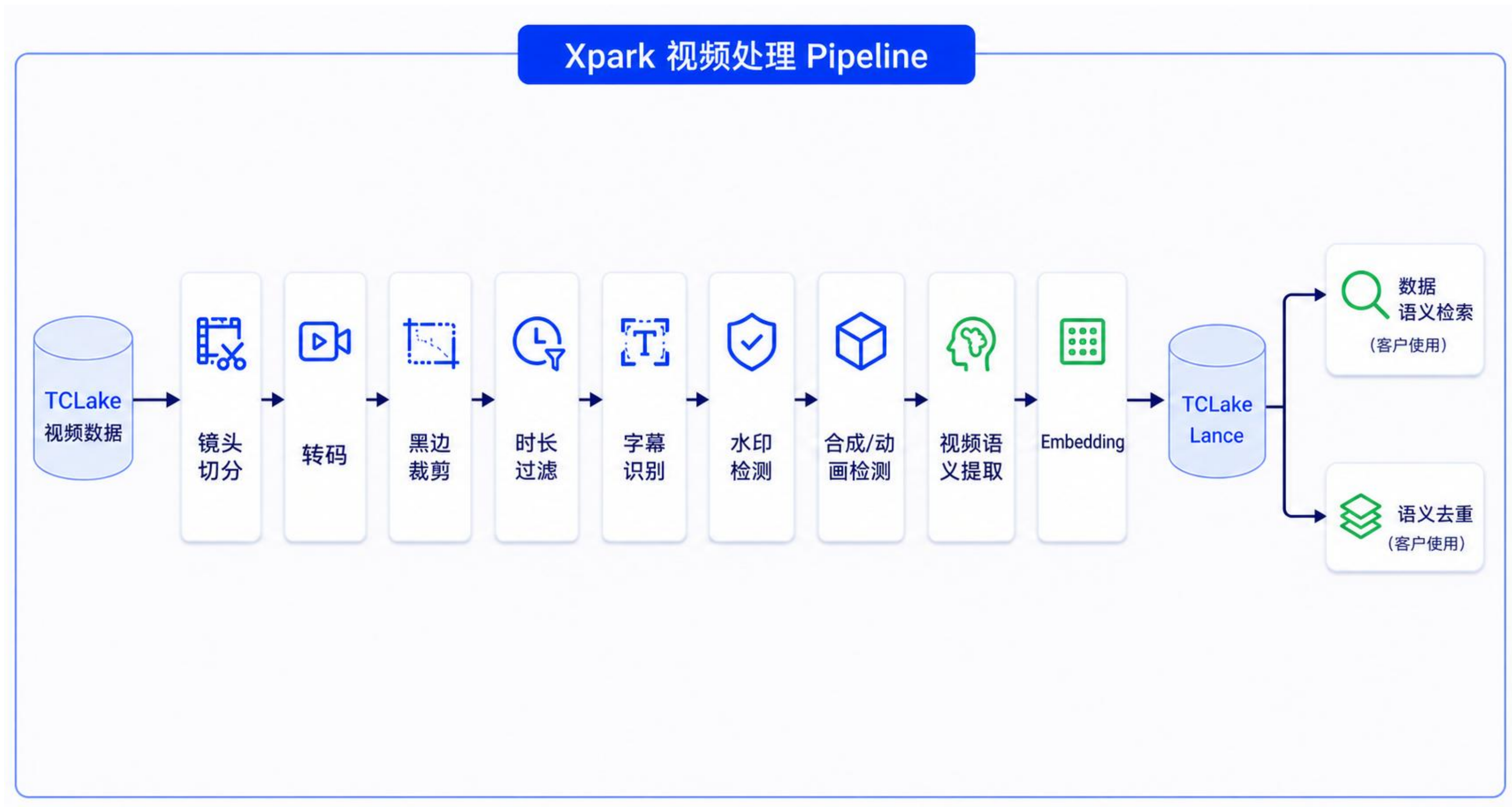
作业配置

作业名称/ID	运行方式	资源对象	作业包	当前任务数	优先级	标签(key:value)	创建人	创建时间	更新时间	操作
xpark_test	按需拉起集群	RayBenchmark/default	xpark_demo.zip	0	5. 中等		100050583949	2026-07-17 17:02:06	2026-07-24 1	编辑 复制作业 运行 更多
test_worker_memory_original	按需拉起集群	RayBenchmark/default	test_worker_memory_reclaim.zip	0	5. 中等		100049539927	2026-07-15 21:26:18	2026-07-15 2	编辑 复制作业 运行 更多
test_worker_memory_reclaim	按需拉起集群	RayBenchmark/default	test_worker_memory_reclaim.zip	0	5. 中等		100049539927	2026-07-15 19:57:21	2026-07-17 1	编辑 复制作业 运行 更多
10_nodes_file_object_store_drago	按需拉起集群	RayBenchmark/default	df_env_test.zip	0	5. 中等		100050523784	2026-07-13 11:49:48	2026-07-20 2	编辑 复制作业 运行 更多
5_nodes_file_object_store_bench	按需拉起集群	RayBenchmark/default	fos_bench.zip	0	5. 中等		100050523784	2026-07-12 20:44:54	2026-07-13 1	编辑 复制作业 运行 更多
10_nodes_file_object_store_bench	按需拉起集群	RayBenchmark/default	fos_bench.zip	0	5. 中等		100050523784	2026-07-10 18:20:58	2026-07-13 1	编辑 复制作业 运行 更多
luke_sleep_while	按需拉起集群	RayBenchmark/default	--	0	5. 中等		100045269314	2026-07-10 18:00:28	2026-07-15 2	编辑 复制作业 运行 更多
5_nodes_file_object_store_env_test	按需拉起集群	RayBenchmark/default	env_test.zip	0	5. 中等		100050523784	2026-07-10 17:34:17	2026-07-13 1	编辑 复制作业 运行 更多
10_nodes_file_object_store_bench	按需拉起集群	RayBenchmark/default	--	0	5. 中等		100050523784	2026-07-10 17:34:17	2026-07-13 1	编辑 复制作业 运行 更多

典型 Pipeline – 用户意图识别



典型 Pipeline – 视频语义知识库构建





AI DLC 数据计算向量化引擎 Meson

为 AI DLC 打造的下一代计算内核

01
—
背景与挑战

02
—
架构介绍

03
—
核心能力

04
—
性能与收益

01

背景与挑战

AI 时代大数据分析面临的挑战

1 业务背景

数据的大规模增长与客户降本诉求

数据规模爆发式增长

AI 与多模态带动计算负载持续增长、*mixed workload* 更复杂，且近实时分析对算力吞吐与响应时延要求显著抬升。

客户降本诉求迫切

算力成本上行而预算趋紧，客户诉求从「堆机器」转向「以更优单位计算成本消化更多负载」。

2 技术挑战

传统计算分析引擎的水土不服

性能: 充分利用硬件

传统计算引擎无法利用现代CPU的向量化和多核能力

IO: 存算分离

云原生架构下，对象存储带来高延迟

全托管调优

系统性能高度依赖人工调参，试错成本极高。
缺乏智能化的自适应调优机制

3 解决方案

数据计算向量化引擎 Meson

高性能向量化算子

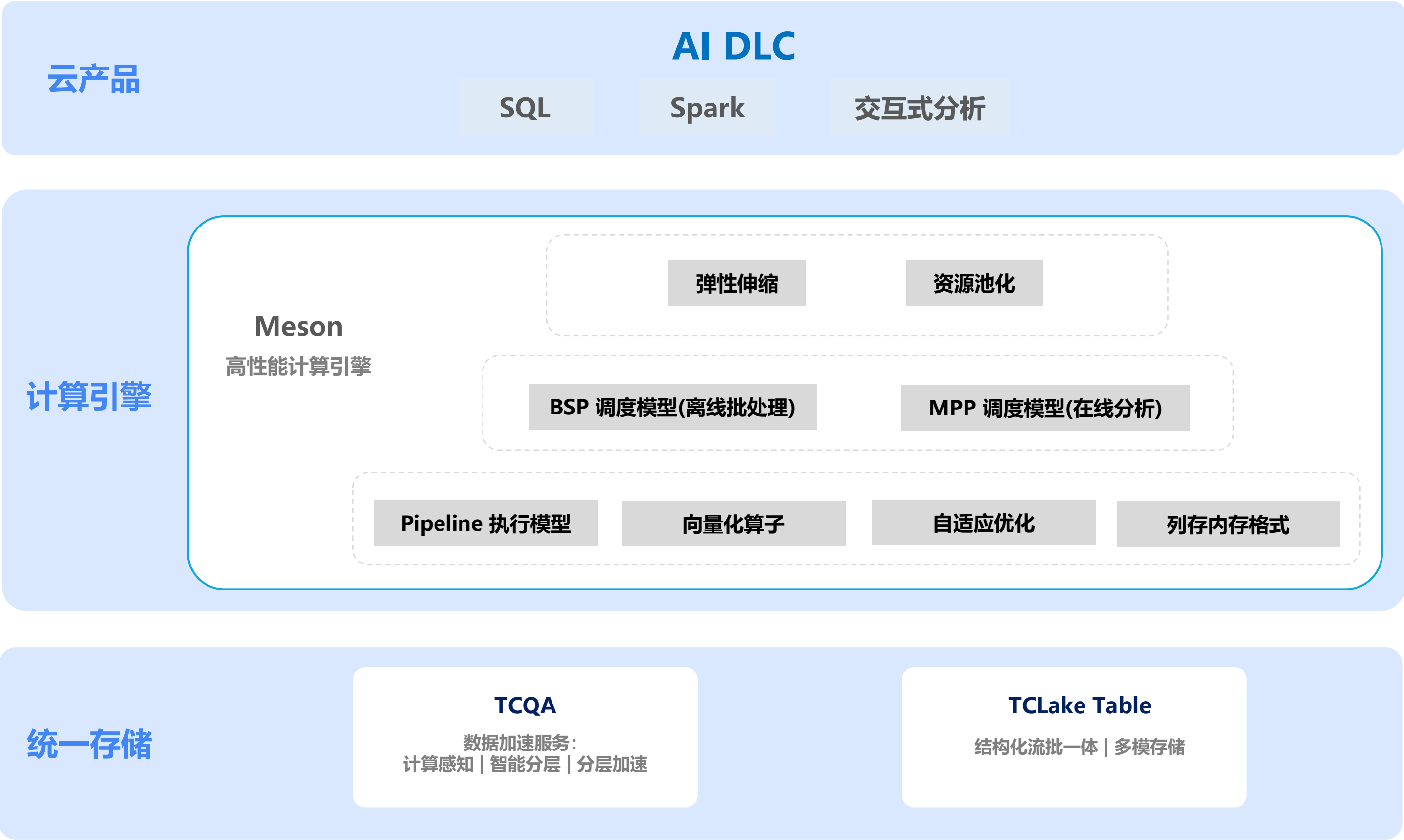
并行执行模型

自适应优化

02

架构介绍

Meson 技术架构



03

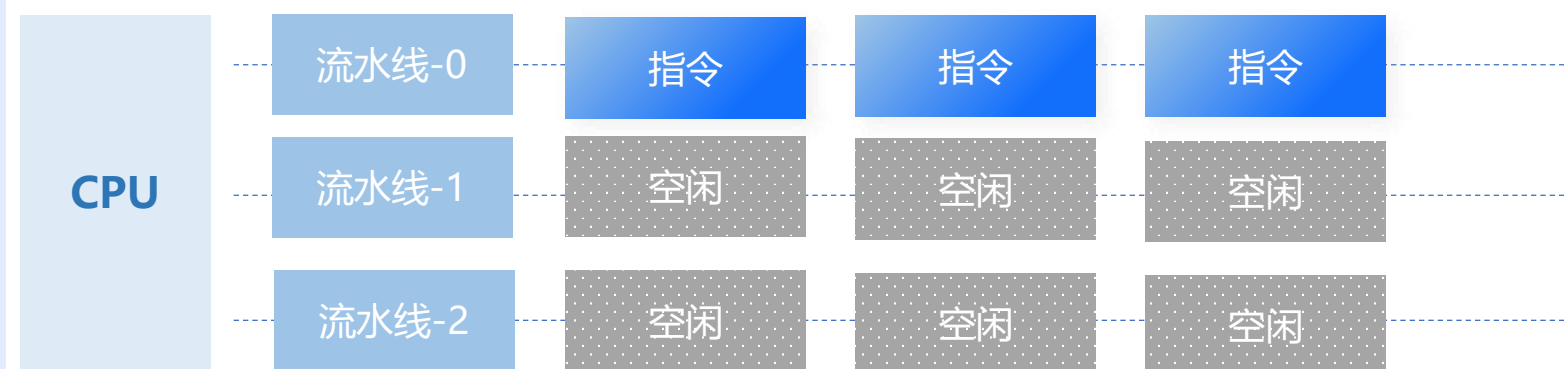
核心能力

核心能力一：向量化算子

现代 CPU 采用超标量流水线技术让单核 CPU 在同一时间执行多条指令，Meson 采用向量化技术提升指令运行吞吐量，压榨 CPU 的极限性能。

典型场景一 CPU 流水线空闲

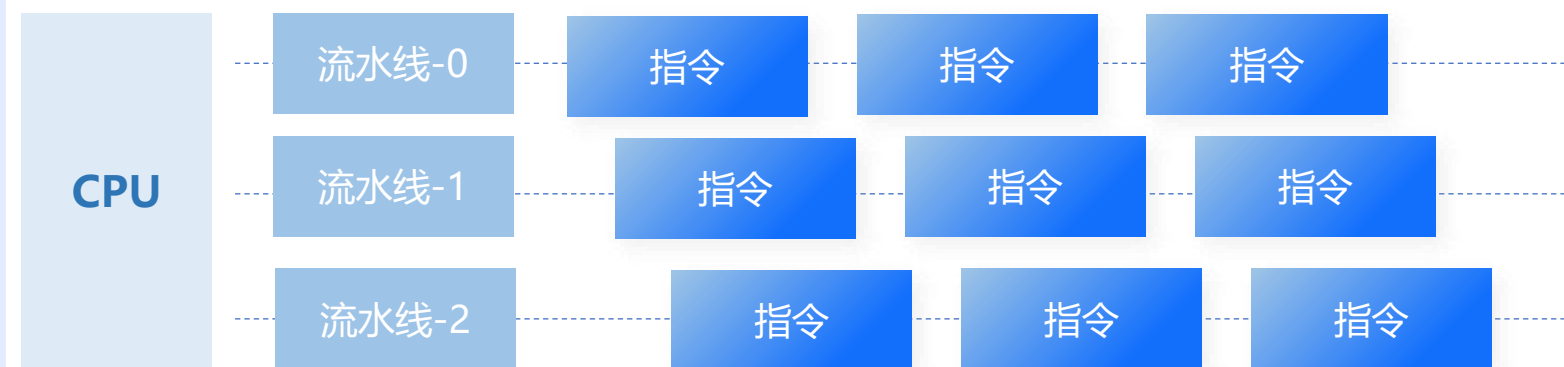
时钟周期



指令串行，流水线空闲：CPU利用率100%，IPC: 0.2

IPC: Instruction Per Clock

时钟周期



指令并行，流水线打满：CPU利用率100%，IPC: 0.6，吞吐提升3x

关联场景向量化优化

常见原因

1. 虚函数调用
2. 分支预测实效
3. 指令间存在数据依赖

```
1 // 标量执行
2 for (data: input) {
3     value = hashTable.find(key);
4     // 判断value是否为空
5     while (value != nullptr) {
6         // 输出关联值
7         output(value);
8         // 关联key等值的value
9         value = value->next;
10    }
11 }
```

```
1 // 批量查找
2 values = hashTable.batchFind(keys);
3 // remove find不上的数据
4 searches = removeMisses(values);
5 while (!searches.empty()) {
6     // output有效的数据
7     batchOutput(searches);
8     // 批量关联key等值的数据
9     values = batchNext(searches);
10    searches = removeMisses(values);
11 }
```

- 1、拆解向量化单元
- 2、消除或减少分支



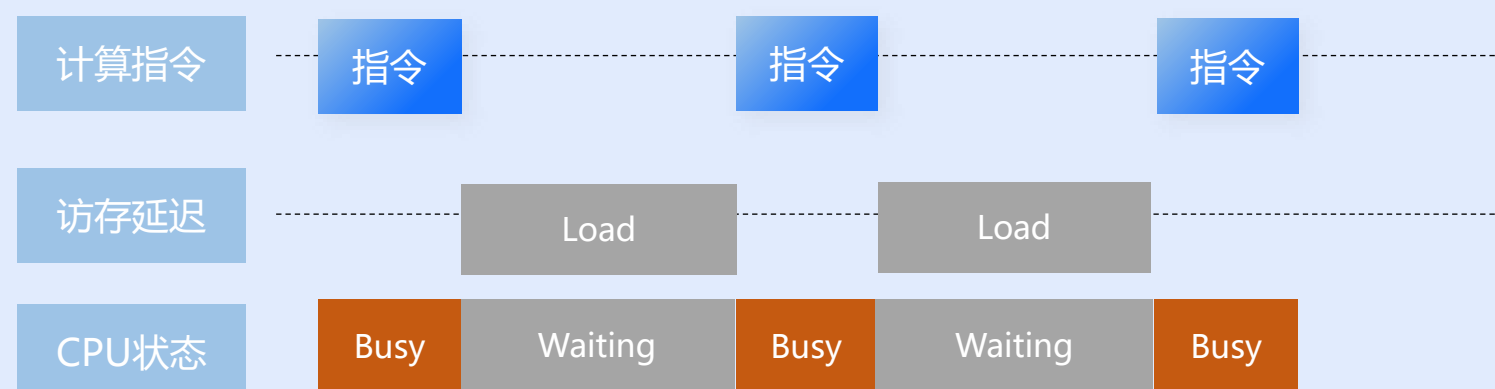
2X+



核心能力一：向量化算子

大数据场景下访存延迟显著阻碍着 CPU 的运行效率，Meson 通过异步将数据加载到缓存，显著提升 CPU 运行效率。

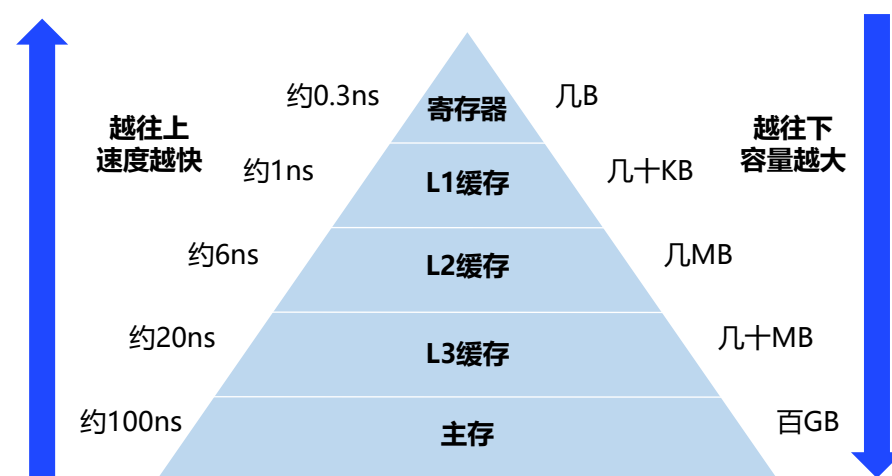
典型场景一 Memory Stalled (内存停顿)



Memory stalled: CPU 利用率**100%**, Busy 态: **<30%**

原因:

- 1、出现CPU Cache Miss
- 2、从主存加载需要100ns+
- 3、引发CPU长时间阻塞



聚合场景 Prefetch 异步访存优化

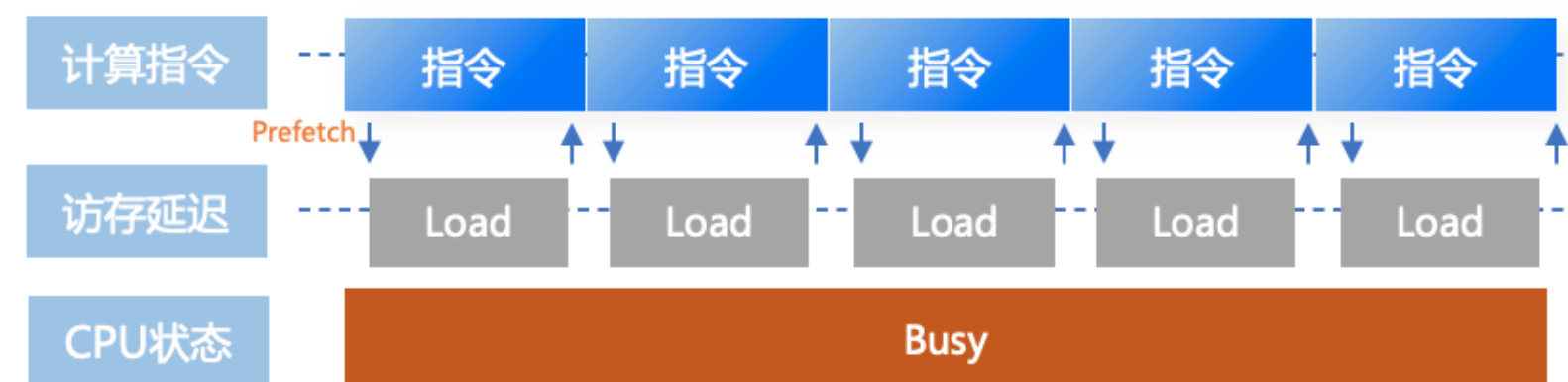
```
1 // 批量计算hash table中位置
2 positions = hash_position(keys);
3
4 for (i = 0 -> num) {
5     // 预取未来第16个元素到L1
6     hashTable.prefetch(positions[i+16]);
7
8     if (hashTable.getKey(positions[i]) == keys[i])
9     {
10         // 聚合计算
11         aggregate(values[i]);
12     }
13 }
```

2.2X+ ↑

Prefetch 原理

1. 内存控制器后台异步加载
2. CPU 和内存并行加载

理想效果: CPU利用率**100%**, Busy态: **约100%**

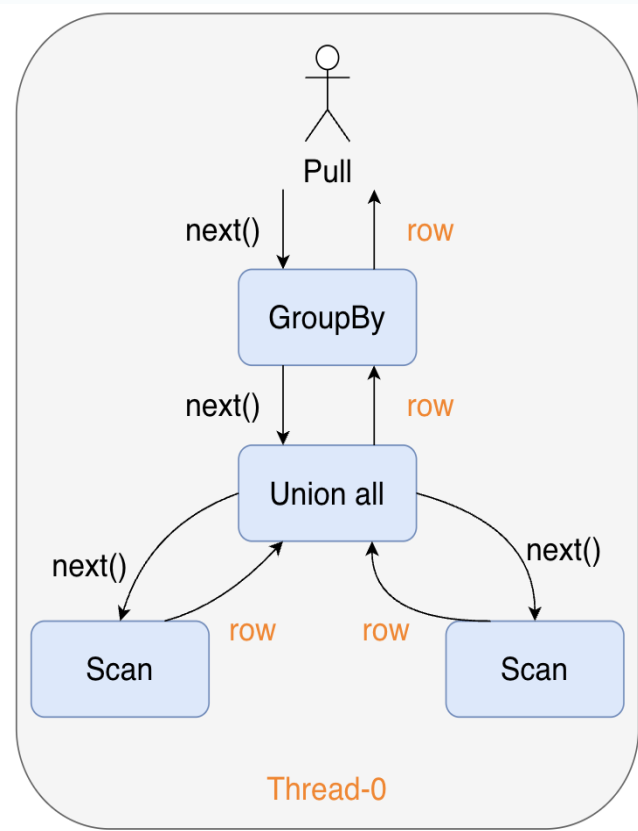


核心能力二: Pipeline 执行模型

Pipeline模型实现计算和数据并行解耦，增加并发能力。

传统volcano模型

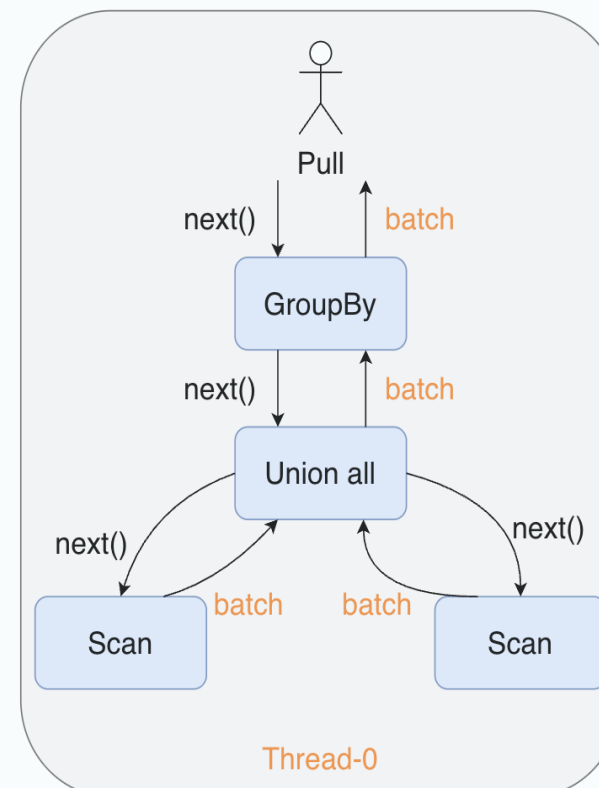
- 1、Row-Based解释执行性能差
- 2、自顶向下Pull驱动，单并发执行



Volcano Model

Vectorized模型

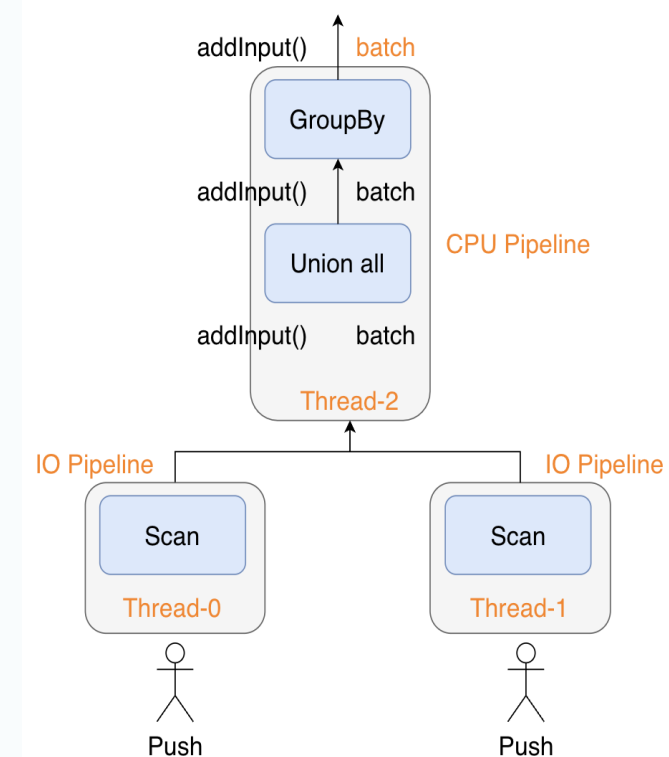
- 1、Batch[4k~8K行]粒度交互，消除解释执行成本
- 2、自顶向下Pull驱动，单并发执行



Vectorized Model

Pipeline模型

- 1、Batch[4k~8K行]粒度交互，消除解释执行成本
- 2、自底向上Push，多并发执行
- 3、拆分CPU、IO Pipeline，计算与数据并行度解耦



Push-based Pipeline Model

解决存算分离瓶颈，重塑数据访问效能

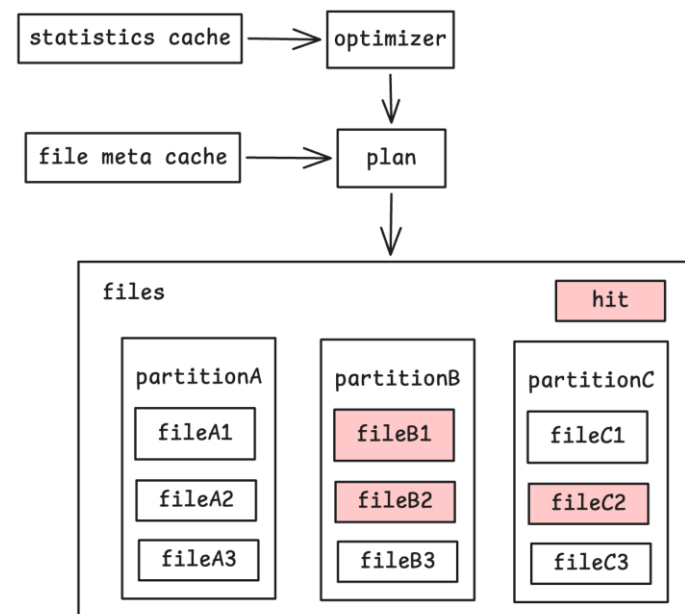
问题与挑战 (元数据)

- 1 无效数据访问带来大量的带宽浪费
[基于元数据可以提前裁剪数据]
- 2 元数据访问本身构成了带宽瓶颈
[海量分区/文件数使元数据请求量膨胀]

解决方案

数据裁剪：利用元数据，精准跳过无效数据块

元数据缓存：加速文件寻址以及优化器决策



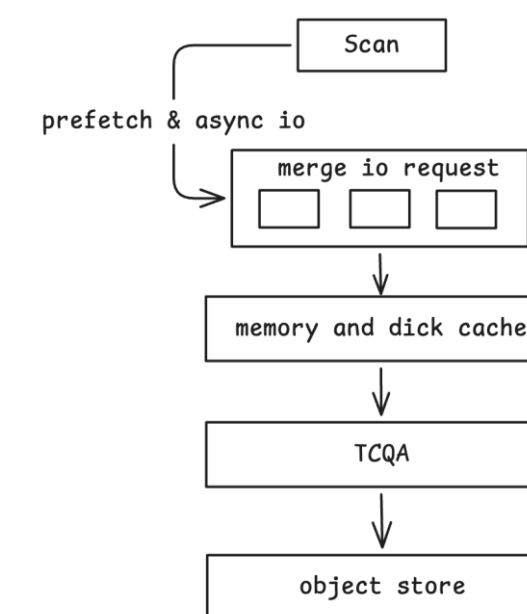
问题与挑战 (数据访问)

- 1 等待 IO 完成导致 CPU 执行卡顿
[CPU 需要等待 IO 完成获取数据后才能开始计算]
- 2 对象存储的高延迟导致查询延迟
[对象存储的延迟远高于本地数据访问]

解决方案

IO 异步和 prefetch：让 CPU 和 IO 并行化

IO 合并和数据缓存：最大化避免低效的对象存储访问



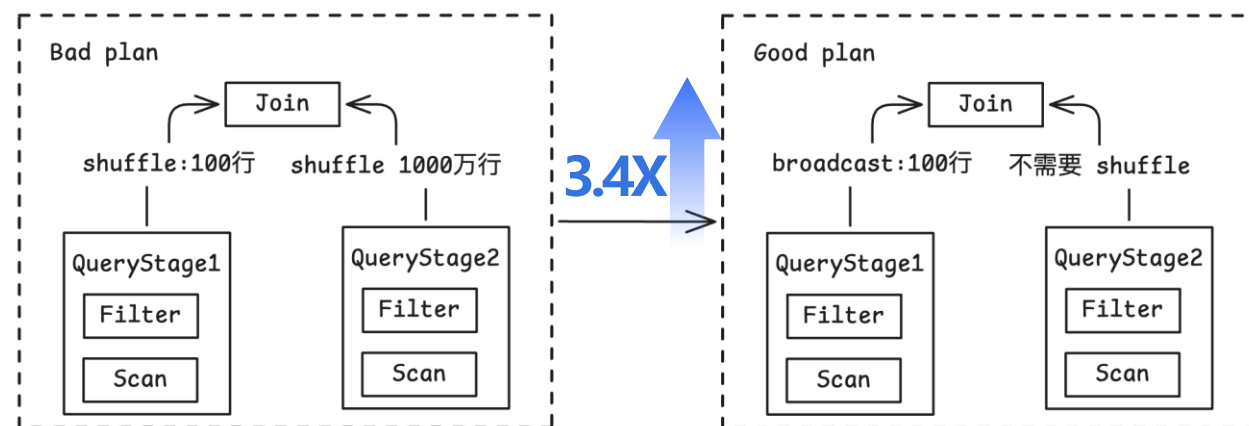
解决全托管调优成本

问题与挑战

- 1 静态配置难以适配多变的 OLAP 负载
[workload 多样且动态变化]
- 2 全托管场景下人工调优成本过高、经验难以沉淀
[人工调优运维负担重]

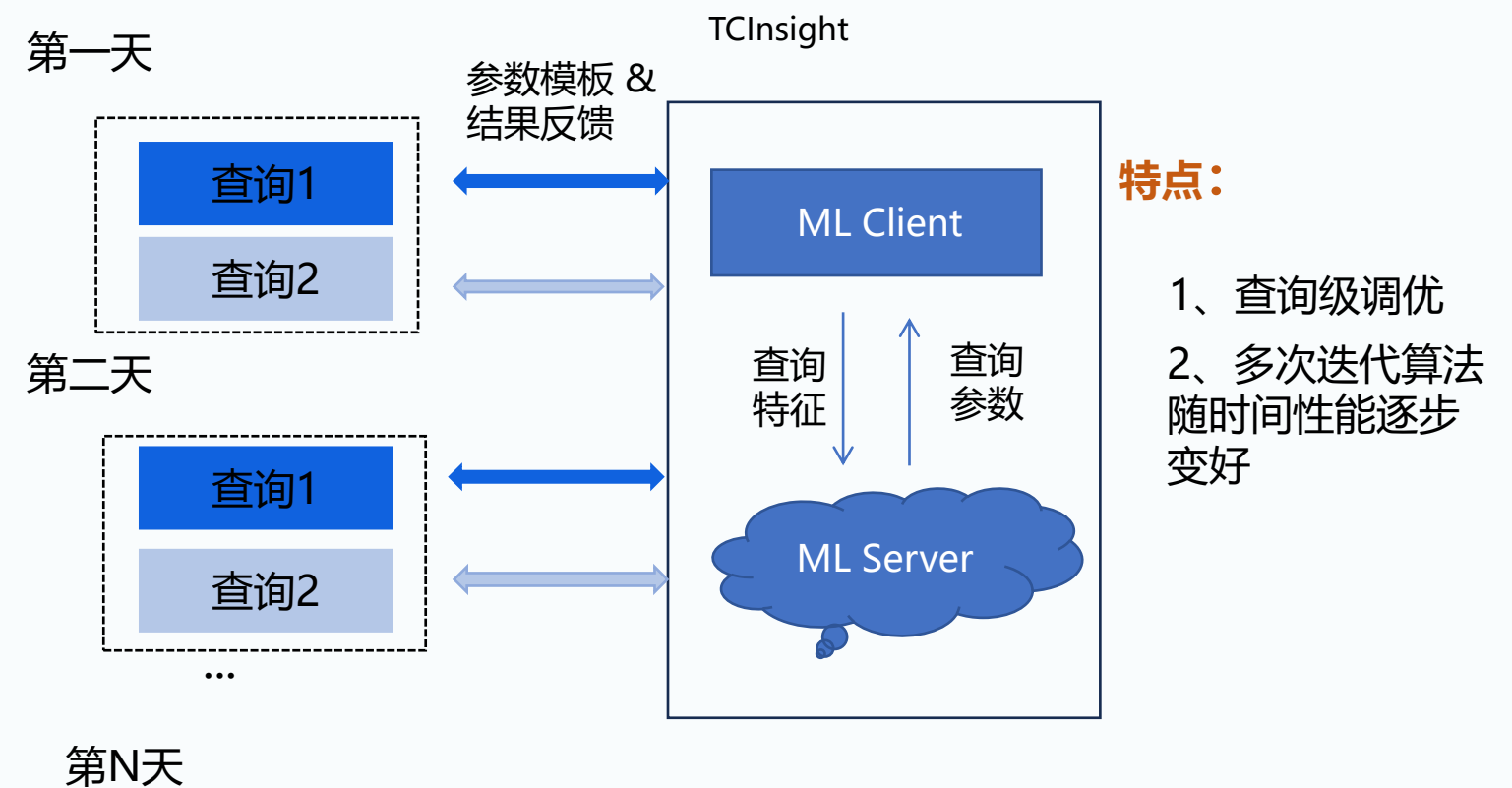
自适应查询优化

- **动态调整**: 运行时优化并发度(大宽表、高压缩率、小文件)与 Shuffle 分区数
- **智能决策**: 自动切换 Join 策略 (HashJoin、SortMergeJoin) 与 Broadcast 阈值
- **实时优化**: adaptive partial agg



TCInsight 智能调优, 进一步提升性能

➢ 基于传统 ML 模型, 实现系统自动检测、查询分析和调优等任务, 无需人为干预



智能性能调优效果

50%人力投入下降

14%性能提升

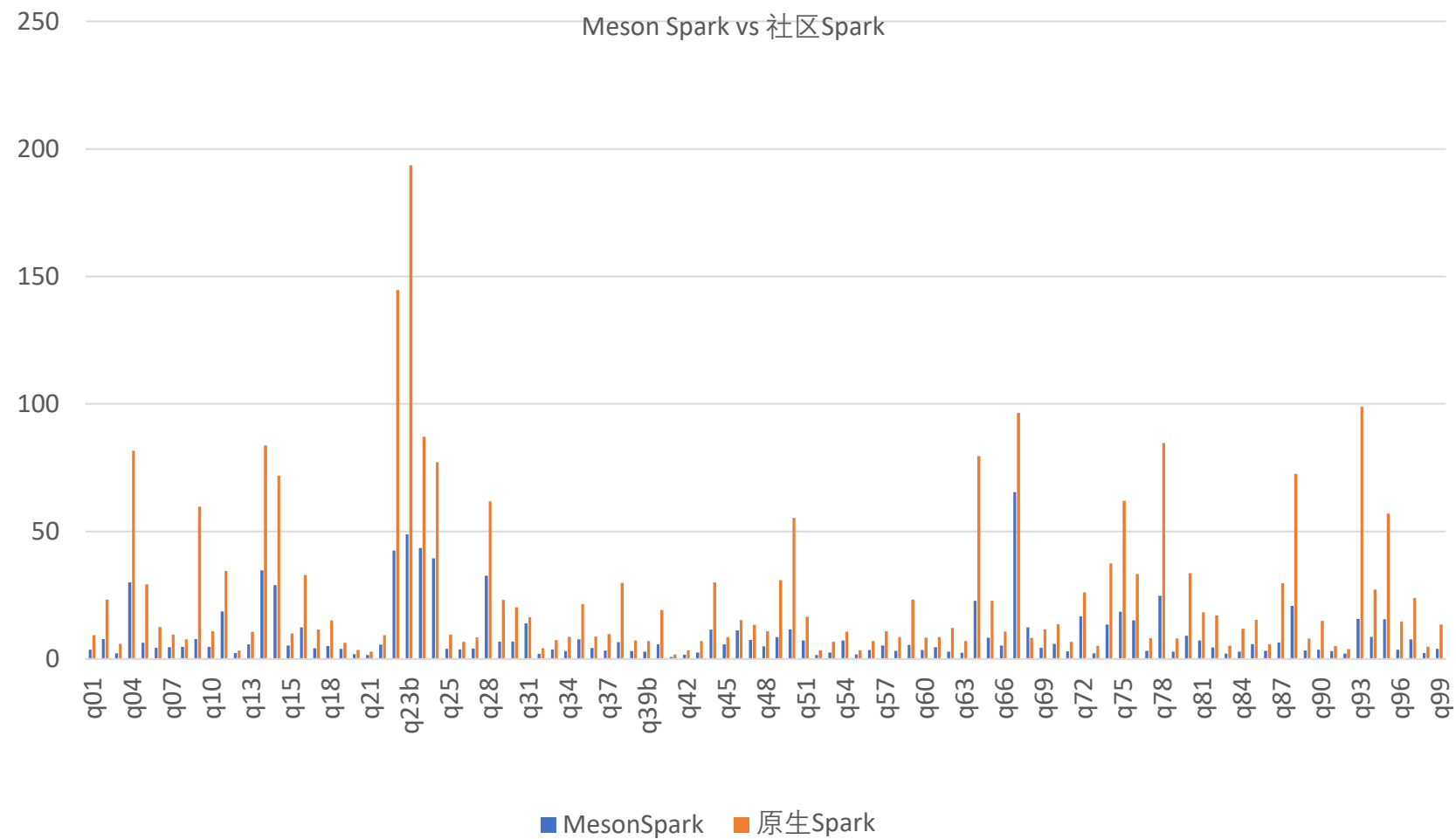
调优人效下降 (9 人天->4 人天)

数据来源: 某客户集群真实负载

04 性能与收益

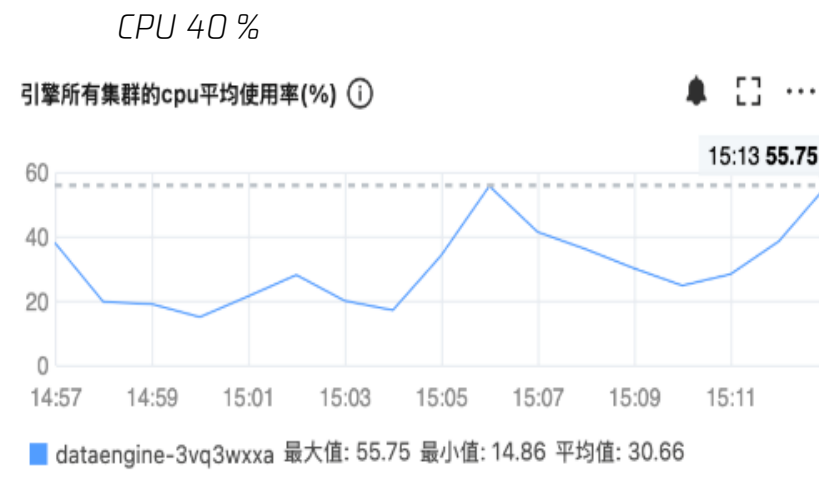
1TB TPCDS 性能提升 3.6X, CPU 消耗下降 1X, 由 CPU 变为 IO 瓶颈

Meson TPCDS 1TB 相较社区Spark 整体性能提升提升3.6X
q23, q24, q64, q78 等计算密集型提升5X

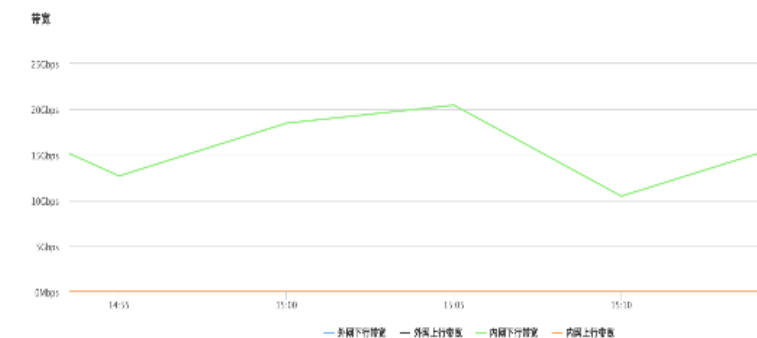


在整体性能提升3.6X的情况下
Meson spark CPU 消耗下降1X, 集群IO吞吐提升近1X
Meson 向量化提升CPU效率显著, 瓶颈点由CPU转向IO

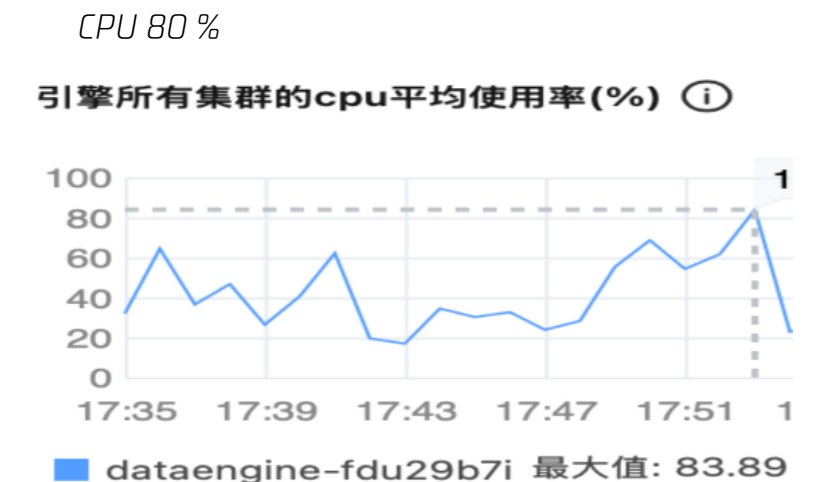
Meson Spark



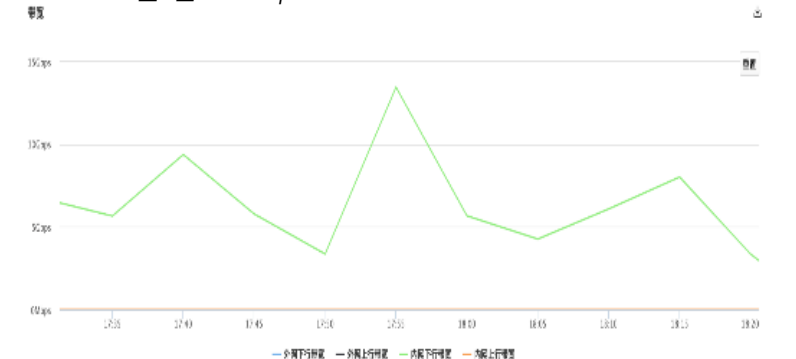
IO吞吐 20Gbps



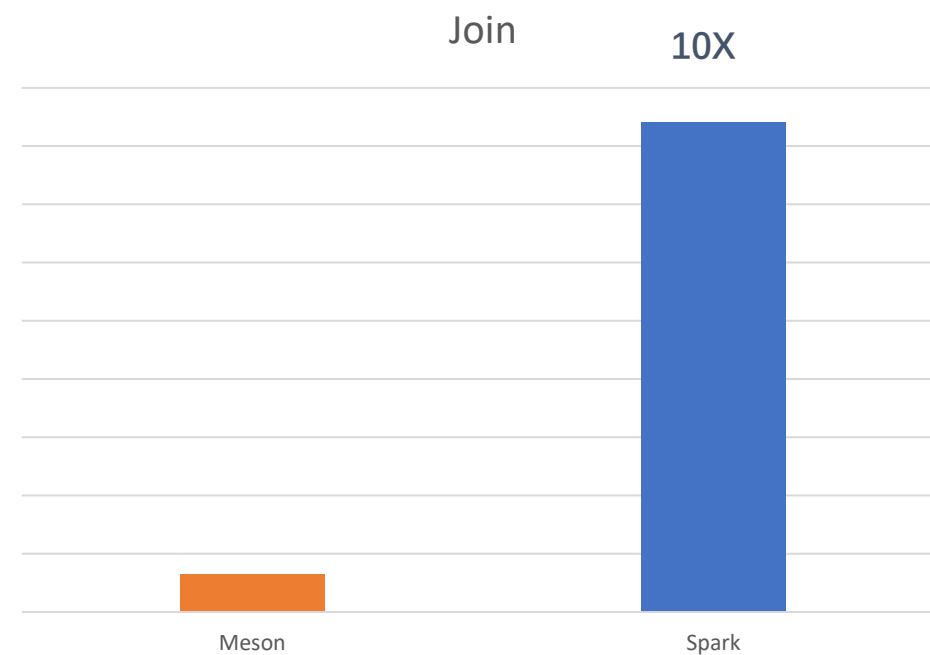
社区Spark



IO吞吐 12Gbps

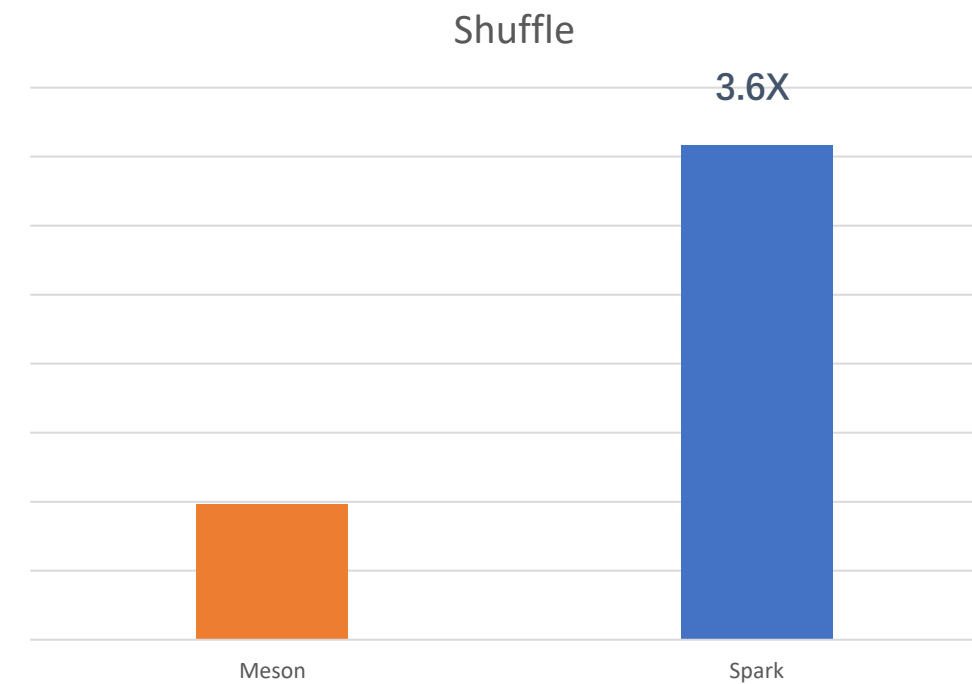


AGG、Join 计算密集型场景提升 3~10X，Scan Shuffle 场景也有显著效果



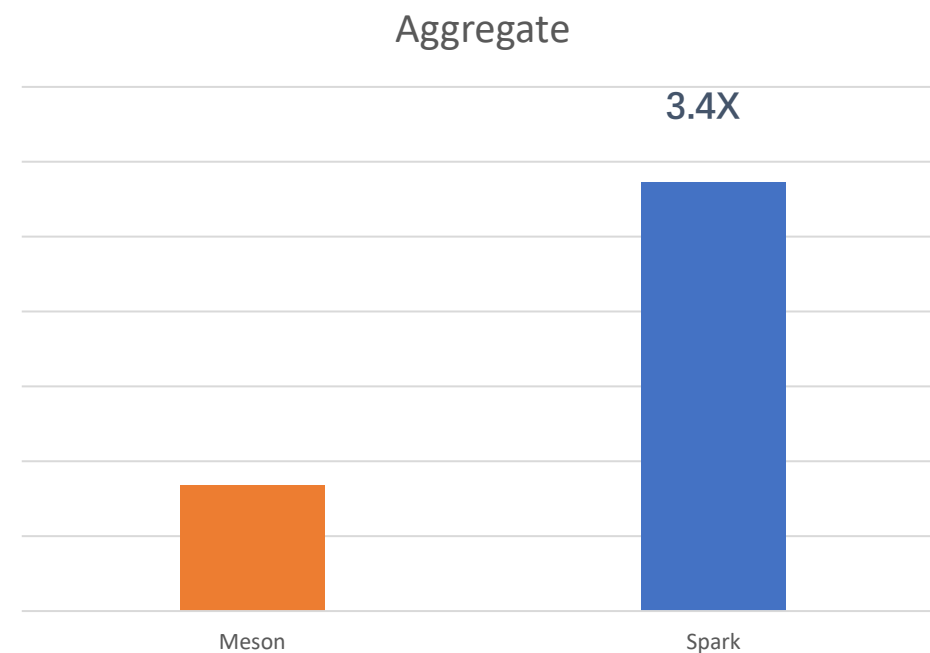
场景:
dim join大表2亿7000万

优化点:
(1).数据layout优化
(2).算子算法优化
(3).Prefetch与访存优化



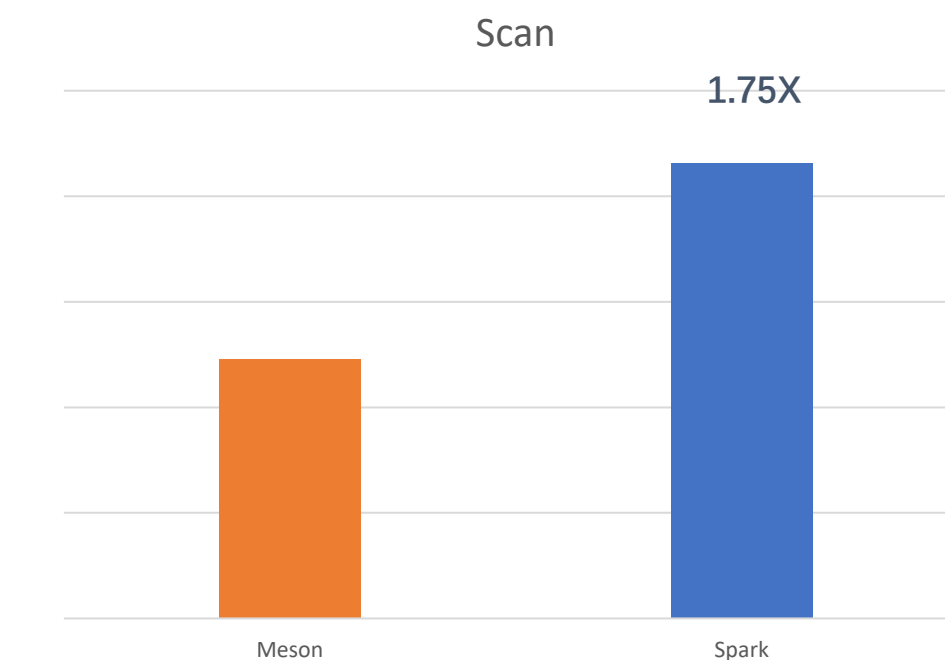
场景:
2亿8千万数据200分区

优化点:
(1).列式压缩
(2).向量化分区



场景:
低聚合度,多列3亿7000万数据

优化点:
(1).数据layout优化
(2).算子算法优化
(3).Prefetch与访存优化



场景:
4列, 700万数据

优化点:
(1).SDD Cache
(2).列式读取

THANKS

感谢您的聆听!



扫码了解更多

2026.07.25